

# **EXPLORING JAVASCRIPT CLOSURES: PRACTICAL USE CASES UNVEILED**

---

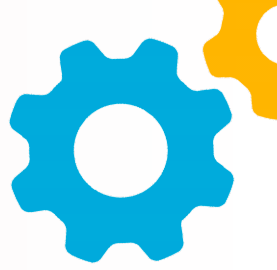


**@DimpleKumari**

Forming a network of fantastic coders.



# WHAT IS CLOSURE?



A **closure** is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment). It gives you access to an outer function's scope from an inner function.

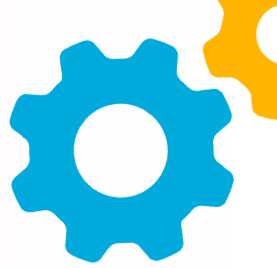
```
function outer() {  
  let x = 10;  
  function inner() {  
    console.log(x);  
  }  
  return inner;  
}  
  
let func = outer();  
func();    //10
```



@DimpleKumari

Forming a network of fantastic coders.





## USE CASE 1 : ENCAPSULATING VARIABLES

---

Closures can be used to **encapsulate variables**, protecting them from external interference while still allowing access and manipulation through the returned function.



@DimpleKumari

Forming a network of fantastic coders.



```
function createBankAccount(initialBalance) {
```

```
  let balance = initialBalance;
```

```
  return {
```

```
    deposit: function(amount) {
```

```
      balance += amount;
```

```
      return balance;
```

```
    },
```

```
    withdraw: function(amount) {
```

```
      if (amount <= balance) {
```

```
        balance -= amount;
```

```
        return balance;
```

```
      } else {
```

```
        return "Insufficient funds";
```

```
      }
```

```
    },
```

```
    checkBalance: function() {
```

```
      return balance;
```

```
    }
```

```
  };
```

```
}
```

```
let account = createBankAccount(100);
```

```
console.log(account.deposit(50)); // Output: 150
```

```
console.log(account.withdraw(20)); // Output: 130
```

```
console.log(account.checkBalance()); // Output: 130
```

## USE CASE 1 : ENCAPSULATING VARIABLES

---



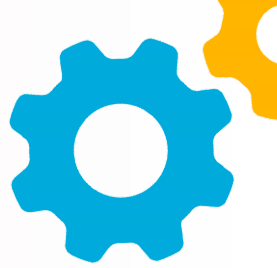
In this example, **the createBankAccount** function creates an object that includes three closures: **deposit**, **withdraw**, and **checkBalance**. These closures all have access to and can operate on the **initialBalance** variable, which is hidden from the outside. The account's balance is encapsulated within the closures, and interactions with the balance are only allowed through the methods provided, thereby achieving encapsulation.



@DimpleKumari

Forming a network of fantastic coders.





## USE CASE 2 : MODULAR DEVELOPMENT

---

Closures play an important role in **modular development**. With closures, we can create private variables and methods, avoiding global namespace conflicts and variable pollution.



@DimpleKumari

Forming a network of fantastic coders.



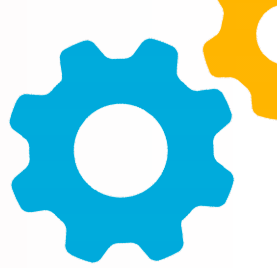
```
const myModule = (function() {
  let privateVariable = 'Private';

  function privateFunction() {
    console.log('Accessed private function');
  }

  return {
    publicFunction: function() {
      console.log('Accessed public function');
    }
  };
})();

myModule.publicFunction();
// Output: Accessed public function
// myModule.privateFunction();

// This would throw an error: Uncaught TypeError: myModule.
privateFunction is not a function
```



## USE CASE 2 : MODULAR DEVELOPMENT

---

In this example, we use an Immediately Invoked Function Expression (**IIFE**) to create an anonymous function scope and return an object with public methods. The **privateVariable** and **privateFunction** defined within the function scope are private, and cannot be directly accessed from the outside.



@DimpleKumari

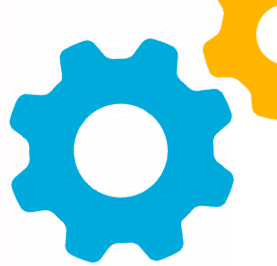
Forming a network of fantastic coders.





## USE CASE 3 : IMPLEMENTING FUNCTION CURRYING

---



**Currying** is a technique in functional programming where a function with multiple arguments is transformed into a sequence of functions, each taking a single argument. This simplifies the function invocation process. Closures can effectively implement function currying.



**@DimpleKumari**

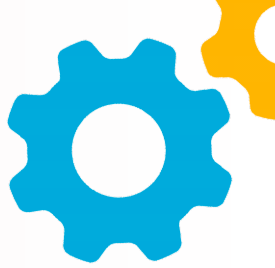
Forming a network of fantastic coders.



```
function createCurry(func) {  
  return function curried(...args) {  
    if (args.length >= func.length) {  
      return func.apply(this, args);  
    } else {  
      return function(...nextArgs) {  
        return curried.apply(this, args.concat(nextArgs));  
      };  
    }  
  };  
}
```

```
function sum(a, b) {  
  return a + b;  
}
```

```
const curriedSum = createCurry(sum);  
console.log(curriedSum(1)(2)); // Output: 3  
console.log(curriedSum(1, 2)); // Output: 3
```



## USE CASE 3 : IMPLEMENTING FUNCTION CURRYING

---

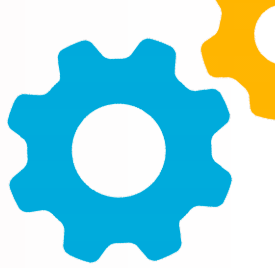
In this example, we've created a generic currying function using **createCurry**. It takes a function as a parameter and returns a curried version of that function. This curried function can be invoked either all at once or partially, allowing for the reuse and flexible invocation of the function.



@DimpleKumari

Forming a network of fantastic coders.

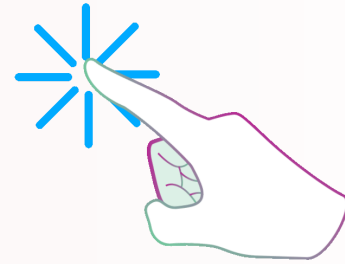




## USE CASE 4: IMPLEMENTING CACHING

---

**Closures** can be used to implement data caching, which is particularly useful for improving performance in situations where functions are called frequently.



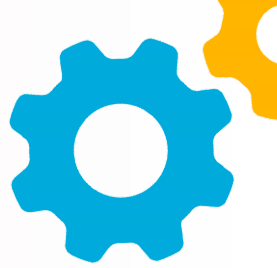
@DimpleKumari

Forming a network of fantastic coders.



```
function createCache() {  
  const cache = new Map();  
  
  return function(key, value) {  
    if (typeof value !== 'undefined') { // Setting cache  
      cache.set(key, value);  
    } else { // Retrieving from cache  
      return cache.get(key);  
    }  
  };  
}
```

```
const cache = createCache();  
cache('name', 'Tom'); // Setting cache  
console.log(cache('name')); // Output: Tom
```



## USE CASE 4: IMPLEMENTING CACHING

---

In this example, we create a **cache** function through the **createCache** function. When we set data in the cache using the cache function, the data is stored in the **cache** Map. When retrieving data from the cache, the cache function looks up and returns the data from the **cache** Map.



@DimpleKumari

Forming a network of fantastic coders.





## USE CASE 5: IMPLEMENTING PRIVATE METHODS

---

For object-oriented programming, **private methods** are an important way to encapsulate data and behavior, preventing external direct access and modification of internal state. **Closures** help us implement private methods.



@DimpleKumari

Forming a network of fantastic coders.



```
function createPerson(name) {  
  let privateGreeting = 'Hello, ' + name;  
  
  return {  
    sayHello: function() {  
      console.log(privateGreeting);  
    }  
  };  
}
```

```
const person = createPerson('Tom');  
person.sayHello(); // Output: Hello, Tom
```

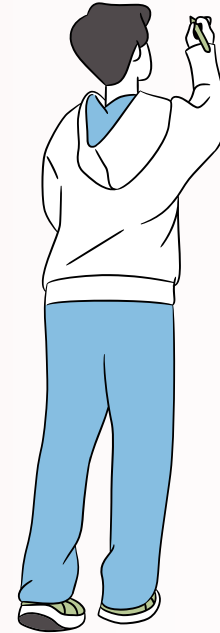




## USE CASE 5: IMPLEMENTING PRIVATE METHODS

---

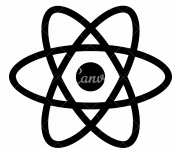
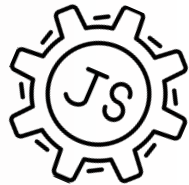
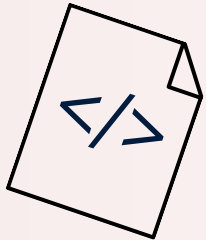
In this example, we create an object with a **createPerson** function, which includes a closure function **sayHello** that can access and operate on the **privateGreeting** variable. The **privateGreeting** variable cannot be directly accessed from the outside, thus ensuring the privacy of the data.



@DimpleKumari

Forming a network of fantastic coders.





FOLLOW



# DIMPLE KUMARI



Forming a network of fantastic coders.

